

Deep Learning

Theory & Interview Mastery

Complete Guide: Foundations → Advanced → Interview Q&A

Perceptrons · CNNs · RNNs · LSTMs · Attention · Transformers · GANs · VAEs · RL · MLOps

• Mathematical Foundations	• Neural Network Basics
• Activation Functions	• Loss Functions & Optimizers
• CNNs & Computer Vision	• RNNs, LSTMs & GRUs
• Attention & Transformers	• GANs & Generative Models
• Regularization Techniques	• Transfer Learning
• Self-Supervised Learning	• Reinforcement Learning
• MLOps & Deployment	• 500+ Interview Q&A;

Table of Contents

- Ch 1.** Mathematical Foundations
- Ch 2.** Neural Network Fundamentals
- Ch 3.** Activation Functions
- Ch 4.** Loss Functions
- Ch 5.** Optimizers & Learning Rate
- Ch 6.** Backpropagation & Gradients
- Ch 7.** Regularization Techniques
- Ch 8.** Convolutional Neural Networks (CNNs)
- Ch 9.** Recurrent Networks: RNN, LSTM, GRU
- Ch 10.** Attention Mechanisms
- Ch 11.** Transformers
- Ch 12.** Transfer Learning & Fine-Tuning
- Ch 13.** Generative Models: GANs & VAEs
- Ch 14.** Self-Supervised & Contrastive Learning
- Ch 15.** Reinforcement Learning Basics
- Ch 16.** Batch Normalization & Normalization Layers
- Ch 17.** Embeddings & Representation Learning
- Ch 18.** Object Detection & Segmentation
- Ch 19.** NLP & Language Models
- Ch 20.** MLOps, Deployment & Scaling
- Ch ★.** Interview Q&A; — All Topics Combined

Chapter 1

Mathematical Foundations

1.1 Linear Algebra

Vectors, matrices, and tensors form the backbone of all deep learning computations. A tensor of rank-n generalizes scalars (rank-0), vectors (rank-1), and matrices (rank-2).

- Matrix multiplication: $(AB)_{ij} = \sum_k A_{ik} B_{kj}$
- Eigendecomposition: $A = Q \Lambda Q^{-1}$ — used in PCA, spectral methods
- SVD: $A = U \Sigma V^T$ — used in dimensionality reduction
- Norms: $L1 = \sum |x_i|$, $L2 = \sqrt{\sum x_i^2}$, Frobenius for matrices
- Dot product and cosine similarity for measuring vector similarity

1.2 Calculus & Derivatives

Chain rule is the mathematical core of backpropagation. If $z=f(g(x))$, then $dz/dx = (dz/dg)(dg/dx)$.

Gradient: $\nabla_w L = [dL/dw_1, dL/dw_2, \dots, dL/dw_n]$

Jacobian: $J_{ij} = df_i/dx_j$ (maps input sensitivity to output)

Hessian: $H_{ij} = d^2f/dx_i dx_j$ (second-order curvature)

- Partial derivatives extend gradients to multi-variable functions
- Directional derivative: rate of change in direction $u = \nabla_f \cdot u$
- Taylor expansion: $f(x+dx) \sim f(x) + \nabla_f dx + 0.5 dx^T H dx$

1.3 Probability & Statistics

Deep learning models are fundamentally probabilistic machines. Key distributions to know:

Distribution	Deep Learning Role
Gaussian $N(\mu, \sigma^2)$	Used in weight init, VAE latent space, noise models
Bernoulli $Bern(p)$	Binary classification output
Categorical $Cat(p)$	Multi-class classification (softmax output)
Uniform $U(a,b)$	Random initialization, dropout masks
Laplace $Lap(\mu, b)$	L1 regularization prior

Bayes theorem: $P(A|B) = P(B|A)P(A) / P(B)$

KL divergence: $KL(P||Q) = \sum P(x) \log[P(x)/Q(x)] \geq 0$

- MLE: maximize $\log P(\text{data}|\theta)$ — gives cross-entropy loss for classification
- MAP: maximize $\log P(\text{data}|\theta) + \log P(\theta)$ — adds regularization
- Entropy $H(X) = -\sum p(x) \log p(x)$ — measures uncertainty
- Mutual information $I(X;Y) = H(X) - H(X|Y)$ — measures shared info

1.4 Information Theory

Cross-entropy: $H(p, q) = -\sum p(x) \log q(x)$

Cross-entropy = entropy + KL divergence: $H(p, q) = H(p) + KL(p || q)$

■ *Tip: Cross-entropy loss = negative log-likelihood under categorical distribution. Minimizing it = maximizing likelihood.*

Chapter 2

Neural Network Fundamentals

2.1 Perceptron & MLP

A perceptron computes $y = f(w \cdot x + b)$. Multiple layers stacked = MLP (Multi-Layer Perceptron). With nonlinear activations, MLPs are universal function approximators (Universal Approximation Theorem).

Forward pass: $a[l] = f(W[l] a[l-1] + b[l])$, $l = 1 \dots L$

- Input layer: raw features x
- Hidden layers: learned representations
- Output layer: task-specific (sigmoid/softmax/linear)
- Width: neurons per layer | Depth: number of layers
- Parameters = $\sum_l (W[l] + b[l])$ shapes

2.2 Universal Approximation Theorem

A single hidden layer MLP with sufficient neurons can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to arbitrary accuracy. BUT: depth is exponentially more efficient than width for many function classes.

■ *Tip: Deeper networks learn hierarchical representations — lower layers capture edges/textures, higher layers capture semantics.*

2.3 Computational Graph

Neural networks are directed acyclic graphs (DAGs). Each node is an operation; edges carry tensors. Forward pass computes outputs; backward pass (autodiff) computes gradients via chain rule.

- Static graphs (TensorFlow 1.x): compiled before run, fast but inflexible
- Dynamic graphs (PyTorch, TF2 eager): built on-the-fly, easier to debug
- Gradient tape records operations for reverse-mode autodiff

2.4 Weight Initialization

Init Method	Use Case / Formula
Random Normal $N(0,0.01)$	Small nets; causes vanishing/exploding in deep nets
Xavier/Glorot	Tanh/sigmoid nets; $\text{var} = 2/(\text{fan_in} + \text{fan_out})$
He/Kaiming	ReLU nets; $\text{var} = 2/\text{fan_in}$
Orthogonal	RNNs; preserves gradient norm
Sparse Init	Large nets; few non-zero weights per neuron

■ **Watch:** Poor initialization -> vanishing gradients (weights too small) or exploding gradients (weights too large). He init is default for ReLU networks.

Chapter 3

Activation Functions

3.1 Why Non-linearity?

Without activation functions, stacked linear layers collapse to a single linear transformation. Non-linearities enable learning complex decision boundaries.

3.2 Common Activations

Function	Formula	Notes
Sigmoid	$1/(1+e^{-x})$	Vanishing grad, saturates at extremes, output in (0,1)
Tanh	$(e^x - e^{-x})/(e^x + e^{-x})$	Zero-centered, still saturates, output in (-1,1)
ReLU	$\max(0, x)$	Fast, sparse, dying ReLU problem for $x < 0$
Leaky ReLU	$\max(0.01x, x)$	Fixes dying ReLU; small negative slope
ELU	x if $x > 0$, $a(e^x - 1)$ else	Smooth, negative vals, mean closer to zero
GELU	$x * \Phi(x)$	Used in BERT/GPT, smooth probabilistic gating
Swish	$x * \text{sigmoid}(x)$	Self-gated, smooth, outperforms ReLU on deep nets
Mish	$x * \tanh(\text{softplus}(x))$	Smooth, non-monotonic, strong in vision tasks
Softmax	$e^{x_i} / \sum(e^{x_j})$	Multi-class output, converts logits to probs
Softplus	$\log(1+e^x)$	Smooth ReLU approximation

3.3 Dying ReLU Problem

When input to ReLU is always negative, the neuron outputs zero and its gradient is also zero — it never updates. Leaky ReLU, ELU, and GELU are designed to mitigate this.

■ *Tip: Use ReLU as default. Switch to GELU for transformers, Swish for EfficientNet-style architectures.*

3.4 Vanishing & Exploding Gradients

Sigmoid/tanh saturate: gradient approaches 0 for large $|x|$. When multiplied across many layers, gradients vanish to zero (or explode if weights are large). Solutions:

- Use ReLU-family activations
- Batch Normalization / Layer Normalization
- Residual connections (skip connections)
- Careful weight initialization (Xavier/He)
- Gradient clipping (for RNNs especially)

Chapter 4

Loss Functions

4.1 Regression Losses

Loss	Formula	Notes
MSE (L2)	$\text{mean}((y - \hat{y})^2)$	Sensitive to outliers; differentiable everywhere
MAE (L1)	$\text{mean}(y - \hat{y})$	Robust to outliers; not differentiable at 0
Huber Loss	MSE if $ e \leq \delta$, MAE otherwise	Best of both: robust + smooth
Log-Cosh	$\text{mean}(\log(\cosh(y - \hat{y})))$	Smooth Huber approximation
MAPE	$\text{mean}(y - \hat{y} /y) * 100$	Scale-invariant; problematic near $y=0$

4.2 Classification Losses

Binary Cross-Entropy: $L = -[y \log(p) + (1-y) \log(1-p)]$

Categorical Cross-Entropy: $L = -\sum_c y_c \log(p_c)$

Focal Loss: $L = -\alpha(1-p_t)^\gamma \log(p_t)$ [addresses class imbalance]

Hinge Loss: $L = \max(0, 1 - y * f(x))$ [SVM-style margin loss]

- KL Divergence used in VAE: $KL(q(z|x) || p(z))$
- Triplet Loss: $\max(0, d(a,p) - d(a,n) + \text{margin})$ for metric learning
- Contrastive Loss: $(1-y)*d^2 + y*\max(0, \text{margin}-d)^2$
- ELBO (Evidence Lower Bound) in VAEs: $E[\log p(x|z)] - KL(q||p)$

4.3 Why Cross-Entropy over MSE for Classification?

MSE with sigmoid output gives very small gradients near 0 and 1 (gradient vanishes). Cross-entropy's gradient is simply $(p - y)$, which is large when prediction is wrong. This makes training much faster.

■ *Tip: Numerically stable cross-entropy: use log-sum-exp trick or compute from logits directly (torch.nn.CrossEntropyLoss does this).*

Chapter 5

Optimizers & Learning Rate

5.1 Gradient Descent Variants

Method	Data Used	Properties
Batch GD	Entire dataset per update	Stable but slow, not scalable
SGD	One sample per update	Noisy, can escape local minima
Mini-batch SGD	Batch of k samples	Best tradeoff; standard practice

5.2 Optimizer Zoo

SGD: $w = w - lr * g$

Momentum: $v = beta * v - lr * g$; $w = w + v$

RMSProp: $E[g^2] = rho * E[g^2] + (1-rho) * g^2$; $w -= lr / sqrt(E+eps) * g$

Adam: $m = b1 * m + (1-b1) * g$; $v = b2 * v + (1-b2) * g^2$; $w -= lr * m_hat / (sqrt(v_hat)+eps)$

AdamW: Adam + decoupled weight decay: $w -= lr * (m_hat / (sqrt(v_hat)+eps) + wd * w)$

AdaGrad: $w -= lr / sqrt(G + eps) * g$ [G accumulates all squared gradients]

Optimizer	Key Params	Best For
SGD+Momentum	b1=0.9	CV tasks with LR scheduling; generalizes well
Adam	b1=0.9, b2=0.999, eps=1e-8	Default for most tasks; fast convergence
AdamW	wd=0.01-0.1	Preferred for transformers; proper weight decay
RMSProp	rho=0.99	RNNs; handles non-stationary objectives
LAMB	Layer-wise	Large-batch training (BERT pretraining)

5.3 Learning Rate Schedules

- Step Decay: $lr = lr * decay_factor$ every N epochs
- Cosine Annealing: $lr(t) = lr_min + 0.5(lr_max - lr_min)(1 + \cos(\pi * t / T))$
- Warmup + Cosine: transformer standard — linear warmup then cosine decay
- Cyclical LR (CLR): oscillates between bounds — helps escape local minima
- OneCycle: increases then decreases LR within one cycle
- ReduceLROnPlateau: reduce when metric stops improving

■ *Tip: AdamW + cosine warmup is the standard recipe for training transformers. SGD + momentum + cosine annealing still wins for many vision tasks.*

■ Watch: Too high LR: loss diverges or oscillates. Too low LR: very slow convergence. Use LR range test (fastai) to find optimal range.

Chapter 6

Backpropagation & Gradients

6.1 Backprop Algorithm

Backpropagation applies chain rule on the computational graph in reverse (backward pass). For each layer l :

$$\text{delta}[L] = dL/da[L] * f'(z[L]) \text{ [output layer error]}$$

$$\text{delta}[l] = (W[l+1]^T \text{delta}[l+1]) * f'(z[l]) \text{ [hidden layer error]}$$

$$dL/dW[l] = \text{delta}[l] (a[l-1])^T \text{ [weight gradient]}$$

$$dL/db[l] = \text{delta}[l] \text{ [bias gradient]}$$

6.2 Vanishing & Exploding Gradients

In deep networks, multiplying many Jacobians during backprop causes gradients to shrink exponentially (vanish) or grow exponentially (explode). This is the core training challenge for deep/recurrent nets.

- Vanishing: sigmoid/tanh + deep networks; use ReLU + BN + residuals
- Exploding: large weights; use gradient clipping (clip_grad_norm_)
- LSTM solves vanishing in RNNs via gated memory cell
- ResNets solve vanishing in CNNs via skip connections: $F(x) + x$

6.3 Gradient Checking

Numerically verify analytical gradients by comparing to finite differences:

$$\text{grad_numerical} \sim [L(\theta + \epsilon) - L(\theta - \epsilon)] / (2 * \epsilon)$$

■ Tip: Use gradient checking only for debugging, not training — it's $O(n)$ in parameters.

6.4 Second-Order Methods

Second-order methods (Newton's method, L-BFGS) use curvature information (Hessian) for faster convergence but are expensive for large models.

- L-BFGS: approximates Hessian inverse; good for small models/fine-tuning
- Natural Gradient (NGD): uses Fisher information matrix; theoretically optimal
- K-FAC: efficient natural gradient approximation for deep nets

Chapter 7

Regularization Techniques

7.1 Bias-Variance Tradeoff

Test error = Bias² + Variance + Irreducible Noise

- High bias = underfitting: model too simple to capture patterns
- High variance = overfitting: model memorizes training data
- Goal: find sweet spot via regularization and proper model capacity

7.2 L1 & L2 Regularization

L2 (Ridge/Weight Decay): $L_{\text{reg}} = L + \lambda \cdot \sum(w^2)$

L1 (Lasso): $L_{\text{reg}} = L + \lambda \cdot \sum(|w|)$

Method	Effect
L2 (Weight Decay)	Shrinks weights toward zero; all stay non-zero; encourages small weights
L1 (Lasso)	Induces sparsity; many weights go exactly to zero; feature selection
Elastic Net	L1 + L2 combination; sparse + stable; for correlated features

7.3 Dropout

Randomly zero out neurons with probability p during training. Each forward pass trains a different 'thinned' network. At inference, scale outputs by $(1-p)$ or use inverted dropout (scale during training).

Train: $h_{\text{drop}} = h \cdot \text{Bernoulli}(1-p) / (1-p)$ [inverted dropout]

- Standard $p=0.5$ for fully connected; $p=0.1-0.2$ for conv layers
- MC Dropout: keep dropout active at test time for uncertainty estimation
- DropConnect: drop weights instead of activations
- SpatialDropout: drop entire feature maps in CNNs
- DropPath/StochasticDepth: drop entire residual paths in transformers/ViT

7.4 Data Augmentation

- Image: random crop, flip, color jitter, rotation, cutout, mixup, cutmix
- Text: synonym replacement, back-translation, token masking (BERT-style)
- Audio: time stretch, pitch shift, additive noise, SpecAugment
- MixUp: $x_{\text{mixed}} = \lambda x_i + (1-\lambda)x_j$; interpolate labels too
- CutMix: paste rectangular region from another image; mix labels proportionally
- RandAugment: automatically search augmentation policy from a set of ops

7.5 Early Stopping

Stop training when validation loss stops improving. Restore best checkpoint. Simple and effective — equivalent to L2 regularization under some conditions.

■ *Tip: Use patience=5-20 epochs. Monitor val loss, not train loss.*

7.6 Batch Normalization as Regularization

BN adds noise from mini-batch statistics — acts as a stochastic regularizer. Reduces need for dropout in CNNs. However, BN and dropout can interfere — avoid combining them naively.

Chapter 8

Convolutional Neural Networks (CNNs)

8.1 Convolution Operation

Output size: $\text{floor}((W - K + 2P) / S) + 1$ [W=input, K=kernel, P=pad, S=stride]

Parameters in conv layer: $K*K*C_{in}*C_{out} + C_{out}$ [with bias]

- Receptive field: region of input that influences an output neuron
- Depthwise conv: separate filter per input channel (MobileNet)
- Pointwise conv: 1x1 conv for channel mixing
- Dilated/Atrous conv: insert zeros to expand receptive field without params
- Transposed conv (deconv): learnable upsampling, used in segmentation decoders

8.2 Pooling

- MaxPool: takes max in window — translation invariance, keeps strongest feature
- AvgPool: averages window — smoother, used for global pooling before classifier
- Global Average Pooling (GAP): one value per feature map — replaces FC layers
- Adaptive pooling: specify output size, pool dynamically (torchvision standard)

8.3 CNN Architecture Evolution

Architecture	Depth	Key Innovation
LeNet-5 (1998)	5 layers	First successful CNN; MNIST handwriting
AlexNet (2012)	8 layers	ImageNet breakthrough; ReLU, dropout, GPU
VGG-16/19 (2014)	16-19 layers	3x3 conv stacks; simple, effective baseline
GoogLeNet (2014)	22 layers	Inception modules; 1x1 conv bottlenecks
ResNet (2015)	50-152 layers	Skip connections; solved vanishing gradient
DenseNet (2016)	dense connections	Each layer connects to all subsequent layers
MobileNet V2 (2018)	inverted residuals	Efficient; depthwise separable convs
EfficientNet (2019)	B0-B7	Compound scaling width/depth/resolution
ConvNeXt (2022)	modern CNN	Modernized ResNet matching ViT performance

8.4 Receptive Field & Inductive Biases

CNNs have two key inductive biases: (1) Translation equivariance — feature detectors work the same everywhere. (2) Locality — nearby pixels are more correlated. These make CNNs very data-efficient for images but less flexible than ViT for global context.

■ *Tip: For most CV tasks, EfficientNetB0-B4 or ResNet50 are excellent baselines. Use pretrained ImageNet weights — always.*

Chapter 9

RNNs, LSTMs & GRUs

9.1 Recurrent Neural Networks

```
h_t = tanh(W_hh * h_{t-1} + W_xh * x_t + b_h)
```

```
y_t = W_hy * h_t + b_y
```

RNNs share weights across time steps (parameter efficiency) but suffer from vanishing gradients for long sequences — they 'forget' distant information.

9.2 LSTM — Long Short-Term Memory

LSTMs add a cell state (long-term memory) controlled by three learnable gates:

```
Forget gate: f_t = sigma(W_f [h_{t-1}, x_t] + b_f)
```

```
Input gate: i_t = sigma(W_i [h_{t-1}, x_t] + b_i)
```

```
Cell update: C_tilde = tanh(W_C [h_{t-1}, x_t] + b_C)
```

```
Cell state: C_t = f_t * C_{t-1} + i_t * C_tilde
```

```
Output gate: o_t = sigma(W_o [h_{t-1}, x_t] + b_o)
```

```
Hidden state: h_t = o_t * tanh(C_t)
```

- Cell state C_t : highway for gradient flow — solves vanishing gradient
- Forget gate: learn what to discard from memory
- Input gate: learn what new info to add
- Output gate: learn what to expose as hidden state
- 4x parameters vs vanilla RNN (4 weight matrices)

9.3 GRU — Gated Recurrent Unit

```
Reset gate: r_t = sigma(W_r [h_{t-1}, x_t])
```

```
Update gate: z_t = sigma(W_z [h_{t-1}, x_t])
```

```
New memory: h_tilde = tanh(W [r_t * h_{t-1}, x_t])
```

```
Hidden state: h_t = (1-z_t)*h_{t-1} + z_t*h_tilde
```

- GRU merges forget and input gates into update gate — simpler, faster
- No separate cell state — combines into one hidden state
- Comparable performance to LSTM on most tasks; fewer parameters
- Better for smaller datasets; LSTM better for very long sequences

9.4 Sequence Modeling Variants

- Bidirectional RNN/LSTM: process sequence forward AND backward; concat states
- Stacked RNN: multiple RNN layers; depth improves feature hierarchy

- Seq2Seq: encoder-decoder architecture for machine translation
- BPTT (Backprop Through Time): unroll RNN; truncated BPTT for efficiency

■ *Tip: LSTMs are largely replaced by Transformers for NLP. Still dominant for time-series, streaming data, and tasks requiring explicit memory gating.*

Chapter 10

Attention Mechanisms

10.1 Motivation: Why Attention?

Seq2Seq models compress the entire input into a fixed-size vector — a bottleneck for long sequences. Attention allows the decoder to selectively focus on relevant encoder states at each decoding step.

10.2 Additive Attention (Bahdanau)

$$\text{score}(h_i, s_t) = v^T \tanh(W_1 h_i + W_2 s_t)$$
$$\alpha_{it} = \text{softmax}(\text{score}(h_i, s_t)) \text{ [attention weights]}$$
$$\text{context}_t = \sum_i \alpha_{it} * h_i \text{ [context vector]}$$

10.3 Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) * V$$

- Q (Query): what we're looking for
- K (Key): what each position advertises
- V (Value): what each position contributes
- Scale by $\sqrt{d_k}$: prevents softmax saturation in high dimensions
- Complexity: $O(n^2 d)$ where n =sequence length, d =dimension

10.4 Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$
$$\text{head}_i = \text{Attention}(Q W_i^Q, K W_i^K, V W_i^V)$$

Each head attends to different representation subspaces — some heads learn syntactic relations, others semantic, others coreference. $h=8$ or 16 heads typical.

10.5 Self-Attention vs Cross-Attention

Type	Source of Q,K,V	Use Case
Self-Attention	Q=K=V come from same sequence	Encoder layers; within-sequence dependencies
Cross-Attention	Q from decoder, K,V from encoder	Decoder layers; seq2seq, conditioning
Causal/Masked	Future positions masked to -inf	Autoregressive decoding (GPT)

■ Tip: Attention is all about routing: which positions in the sequence matter for predicting the next token or classifying the input.

Chapter 11

Transformers

11.1 Architecture Overview

Transformer (Vaswani et al., 2017) — fully attention-based, no recurrence. Parallelizable, models long-range dependencies. Two variants: Encoder-only (BERT), Decoder-only (GPT), Encoder-Decoder (T5, BART).

11.2 Encoder Block

$\text{LayerNorm}(x + \text{MultiHeadSelfAttn}(x))$

$\text{LayerNorm}(x + \text{FFN}(x))$ where $\text{FFN} = \text{Linear} \rightarrow \text{ReLU/GELU} \rightarrow \text{Linear}$

- Pre-norm (modern): LayerNorm before sublayer — more stable training
- Post-norm (original): LayerNorm after sublayer + residual
- FFN hidden dim typically 4x model dim (e.g., $d_{\text{model}}=768$, $d_{\text{ff}}=3072$)
- Residual connections throughout — critical for gradient flow

11.3 Positional Encoding

$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d_{\text{model}})})$

$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i/d_{\text{model}})})$

- Sinusoidal PE (original): fixed, generalizes to unseen lengths
- Learned PE (BERT): trainable position embeddings up to max length
- Relative PE (T5, ALiBi): encode relative distance between positions
- RoPE (Rotary Position Embedding): rotate Q,K vectors — LLaMA, GPT-NeoX

11.4 Key Transformer Models

Model	Type	Key Use / Innovation
BERT	Encoder-only	Bidirectional MLM + NSP; classification, NER, QA
GPT-2/3/4	Decoder-only	Causal LM; autoregressive text generation
T5	Encoder-Decoder	Text-to-text; all tasks as generation
ViT	Encoder-only	Patch-based image tokens; image classification
CLIP	Dual encoder	Contrastive image-text pretraining
LLaMA 2/3	Decoder-only	Open LLM; RoPE + GQA + SwiGLU
Mistral	Decoder-only	Sliding window attention + GQA

11.5 Efficient Attention Variants

- Sparse Attention (Longformer): local + global attention — $O(n)$ instead of $O(n^2)$

- Linear Attention: kernel trick to approximate softmax — $O(n)$
- FlashAttention: IO-aware exact attention via tiling — 2-4x faster, same result
- GQA (Grouped Query Attention): few key-value heads shared — LLaMA2, Mistral
- MQA (Multi-Query Attention): single K,V head — very fast inference
- ALiBi: add linear bias to attention scores — length extrapolation without PE

■ *Tip: FlashAttention is the go-to for training transformers today — drop-in replacement with significant memory and speed gains.*

Chapter 12

Transfer Learning & Fine-Tuning

12.1 Why Transfer Learning?

Training from scratch requires massive data and compute. Pretrained models on large datasets (ImageNet, Wikipedia) have learned general representations. Transfer learning adapts these for downstream tasks with less data.

12.2 Transfer Learning Strategies

Strategy	What Trains	Best For
Feature Extraction	Freeze backbone; train only classifier head	Very small datasets
Full Fine-Tuning	Unfreeze all layers; retrain everything	Large downstream dataset
Gradual Unfreezing	Unfreeze layers top-to-bottom progressively	Medium datasets
Two-Phase Training	Phase 1: frozen backbone; Phase 2: unfreeze all	Standard in CV
Adapter Tuning	Insert small trainable adapter modules	Multiple tasks from one base
Prompt Tuning	Learn soft prompts; freeze model	LLMs; parameter-efficient
LoRA	Low-rank updates: $W' = W + AB$	LLMs; ~0.1% parameter overhead

12.3 LoRA — Low-Rank Adaptation

$W' = W + \Delta W = W + A * B$ where $A \in \mathbb{R}^{(d \times r)}$, $B \in \mathbb{R}^{(r \times k)}$, $r \ll \min(d, k)$

- $r=4-16$ typically sufficient; full rank $r=d$ is equivalent to full fine-tuning
- Only A and B are trained; W is frozen — massive parameter savings
- QLoRA: quantize W to 4-bit, train LoRA adapters in float16
- LoRA merges into base model at inference: $W' = W + AB$ — no latency cost

12.4 Domain Adaptation

- Covariate shift: $P_{\text{train}}(x) \neq P_{\text{test}}(x)$; importance weighting helps
- Label shift: $P_{\text{train}}(y) \neq P_{\text{test}}(y)$; recalibrate classifier
- Domain adversarial training: learn features invariant to domain
- Self-training: pseudo-labels on target domain unlabeled data

■ *Tip: For CNN vision tasks: use ImageNet pretrained weights, two-phase training, LR warmup, and AdamW. This recipe works for 90% of CV tasks.*

Chapter 13

Generative Models: GANs & VAEs

13.1 Variational Autoencoders (VAEs)

Encoder: $q_{\phi}(z|x)$ -- approximate posterior

Decoder: $p_{\theta}(x|z)$ -- likelihood

ELBO: $L = E_q[\log p(x|z)] - KL(q(z|x) || p(z))$

Reparameterization: $z = \mu + \epsilon \cdot \sigma$, $\epsilon \sim N(0, I)$ [enables backprop]

- Latent space is continuous and regularized — enables interpolation and sampling
- KL term forces posterior toward $N(0, I)$ — prevents posterior collapse
- Beta-VAE: upweight KL ($\beta > 1$) for more disentangled representations
- VQ-VAE: discrete latent codes via vector quantization — used in DALL-E

13.2 Generative Adversarial Networks (GANs)

$\min_G \max_D E[\log D(x)] + E[\log(1 - D(G(z)))]$

- Generator G: maps noise $z \rightarrow$ fake samples; wants D to output 1
- Discriminator D: distinguishes real vs fake; wants to output 1 for real, 0 for fake
- Nash equilibrium: G generates data distribution; D outputs 0.5 everywhere
- Mode collapse: G produces limited variety — major training challenge
- Training instability: alternating optimization is delicate

GAN Variant	Key Innovation
DCGAN	Deep conv generator + discriminator; stable training baseline
WGAN / WGAN-GP	Wasserstein distance; gradient penalty; much more stable
StyleGAN2/3	Style-based generator; state-of-art high-res faces
CycleGAN	Unpaired image-to-image translation via cycle consistency
Pix2Pix	Paired image translation with L1 + adversarial loss
BigGAN	Class-conditional; self-attention; large-scale ImageNet generation

13.3 Diffusion Models

Current state-of-the-art generative models. Forward process: gradually add Gaussian noise. Reverse process: learn to denoise step by step.

Forward: $q(x_t|x_{t-1}) = N(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$

Reverse: $p_{\theta}(x_{t-1}|x_t) = N(x_{t-1}; \mu_{\theta}(x_t, t), \sigma^2 I)$

- DDPM: 1000 denoising steps; high quality but slow

- DDIM: deterministic sampling; 50-100 steps
- Latent Diffusion (Stable Diffusion): diffuse in latent space; much faster
- Classifier-free guidance: condition without separate classifier

Chapter 14

Self-Supervised & Contrastive Learning

14.1 Self-Supervised Learning

Learn representations from unlabeled data using pretext tasks — no manual labels needed. The learned representations transfer well to downstream tasks.

Pretext Task	Description
Masked Language Modeling	Mask tokens; predict them (BERT)
Causal Language Modeling	Predict next token (GPT)
Masked Image Modeling	Mask patches; predict pixels/features (MAE, BEiT)
Rotation Prediction	Predict image rotation angle (0/90/180/270)
Contrastive Learning	Pull together similar pairs; push apart dissimilar (SimCLR, MoCo)
Jigsaw Puzzle	Predict correct permutation of image patches
I-JEPA	Predict masked patch features from context (no pixel reconstruction)

14.2 Contrastive Learning

InfoNCE Loss: $L = -\log \left[\frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_k \exp(\text{sim}(z_i, z_k)/\tau)} \right]$

- Positive pair: two augmented views of same image
- Negative pairs: all other images in batch
- SimCLR: large batch (4096+) for many negatives; project to lower dim before loss
- MoCo: momentum encoder + memory queue — efficient negatives without large batch
- BYOL: no negatives! Uses target network (EMA) + predictor to avoid collapse
- SimSiam: no negatives, no momentum — stop-gradient prevents collapse
- DINO: self-distillation with centering; emergent segmentation in ViT attention

14.3 Why SSL Works

Augmentation-based SSL forces the model to learn features invariant to nuisances (crop, flip, color) while retaining semantic content. The representation captures the structure that makes two views of the same image similar.

■ *Tip: DINO features from ViT exhibit semantic segmentation for free — attention heads learn to attend to objects without any supervision.*

Chapter 15

Reinforcement Learning Basics

15.1 RL Framework

Agent interacts with environment: observe state s , take action a , receive reward r , transition to next state s' .
Goal: maximize expected cumulative discounted reward.

Return: $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$

Value: $V_{\pi}(s) = E_{\pi}[G_t \mid S_t=s]$

Q-value: $Q_{\pi}(s,a) = E_{\pi}[G_t \mid S_t=s, A_t=a]$

Bellman: $V(s) = E[R + \gamma V(S')]$

15.2 Key Algorithms

Algorithm	Key Idea
Q-Learning	Off-policy; learns $Q^*(s,a)$; tabular/DQN
DQN	Q-learning + CNN + experience replay + target network
Policy Gradient (REINFORCE)	Directly optimize policy π_{θ} via gradient ascent
Actor-Critic (A2C/A3C)	Policy (actor) + value (critic) trained jointly
PPO	Proximal Policy Optimization; clip objective; stable, sample-efficient
SAC	Soft Actor-Critic; entropy maximization; off-policy; continuous actions
RLHF	RL from Human Feedback; reward model from comparisons; fine-tunes LLMs

15.3 Deep Q-Networks (DQN)

$L = E[(r + \gamma \max_{a'} Q_{\text{target}}(s', a') - Q(s, a))^2]$

- Experience Replay: store (s,a,r,s') in buffer; sample mini-batches — breaks correlations
- Target Network: frozen copy of Q network; updated every N steps — stability
- Double DQN: separate selection and evaluation networks — reduces overestimation
- Dueling DQN: $Q(s,a) = V(s) + A(s,a)$ — better value estimation
- Prioritized Experience Replay: sample important transitions more often

■ *Tip: PPO is the most practically reliable RL algorithm. Used to train ChatGPT via RLHF. SAC is preferred for continuous control.*

Chapter 16

Normalization Layers

16.1 Batch Normalization (BN)

```
mu_B = (1/m) sum x_i; sigma^2_B = (1/m) sum (x_i - mu_B)^2
```

```
x_hat = (x - mu_B) / sqrt(sigma^2_B + eps)
```

```
y = gamma * x_hat + beta [gamma, beta: learnable]
```

- Normalizes over batch dimension (N) per feature
- Pros: reduces internal covariate shift, faster training, acts as regularizer
- Cons: batch-size dependent; doesn't work well for small batches or online learning
- Test time: use running mean/variance (EMA from training)

16.2 Normalization Variants

Method	Normalize Over	Stats	Best For
Batch Norm (BN)	N,H,W	Per feature	CNNs; sensitive to batch size
Layer Norm (LN)	H,W,C	Per sample	Transformers, RNNs; batch-size independent
Instance Norm (IN)	H,W per channel	Per sample per channel	Style transfer
Group Norm (GN)	Groups of channels	Per group per sample	Small batches, detection
RMS Norm	Per sample	Root mean square	LLaMA; simpler, no mean centering

■ *Tip: LayerNorm is standard for transformers. BatchNorm for CNNs with batch size ≥ 32 . GroupNorm ($G=32$) for detection/segmentation where batch sizes are small.*

■ **Watch: BN and Dropout interact poorly — their noise scales interfere. Use one or the other. In modern transformers, use LayerNorm + no dropout for small models.**

Chapter 17

Embeddings & Representation Learning

17.1 Word Embeddings

- One-hot: sparse, no semantic meaning, $O(V)$ dimensions
- Word2Vec CBOW: predict center word from context
- Word2Vec Skip-gram: predict context from center word
- GloVe: factorize global co-occurrence matrix
- FastText: sub-word embeddings; handles OOV words
- Contextual (BERT): same word $\rightarrow$ different embedding based on context

17.2 Sentence & Document Embeddings

- Sentence-BERT: siamese BERT; optimized for semantic similarity
- Universal Sentence Encoder: transformer + DAN; multilingual
- CLIP text encoder: image-aligned text embeddings
- Average pooling over token embeddings (simple but effective baseline)

17.3 Image Embeddings

- CNN features: pool5 of ResNet50, GAP output = 2048-d vector
- DINO ViT-B/16 [CLS] token: rich semantic features, unsupervised
- CLIP image encoder: language-aligned; good for zero-shot tasks

17.4 Embedding Space Properties

Cosine similarity: $\cos(u,v) = (u \cdot v) / (||u|| \cdot ||v||)$ in $[-1, 1]$

- King - Man + Woman $\sim$ Queen: linear structure in Word2Vec space
- Interpolation in VAE latent space gives semantically smooth transitions
- Nearest neighbor search in embedding space = semantic retrieval (RAG)
- Dimensionality reduction: PCA, t-SNE, UMAP for visualization

Chapter 18

Object Detection & Segmentation

18.1 Detection Frameworks

Model	Type	Key Innovation
R-CNN family	Two-stage	Selective search + CNN; slow but accurate
Fast R-CNN	Two-stage	ROI pooling on shared feature map
Faster R-CNN	Two-stage	Region Proposal Network (RPN); end-to-end
YOLO v1-v8	One-stage	Grid-based regression; real-time detection
SSD	One-stage	Multi-scale feature maps; default boxes
RetinaNet	One-stage	Focal loss; solves class imbalance in one-stage
DETR	End-to-end	Transformer; no NMS; set prediction with bipartite matching

18.2 Key Concepts

$\text{IoU} = \text{Area}(\text{Pred} \cap \text{GT}) / \text{Area}(\text{Pred} \cup \text{GT})$ [Intersection over Union]

NMS: keep highest confidence box; suppress overlapping boxes ($\text{IoU} > \text{threshold}$)

mAP = mean over classes of Average Precision at IoU threshold(s)

18.3 Segmentation

Type	Goal	Key Models
Semantic Seg.	Label each pixel with class	FCN, DeepLab, SegFormer
Instance Seg.	Separate instances of same class	Mask R-CNN
Panoptic Seg.	Semantic + instance combined	Panoptic FPN, Mask2Former
SAM	Promptable any-object segmentation	ViT + prompt encoder (Meta)

Chapter 19

NLP & Language Models

19.1 Language Model Fundamentals

$P(w_1, \dots, w_n) = \text{product } P(w_i \mid w_1, \dots, w_{i-1})$ [autoregressive]

$\text{Perplexity} = \exp(-(1/N) \sum \log P(w_i \mid \text{context}))$ [lower = better]

- Causal LM (GPT): left-to-right; generate text token by token
- Masked LM (BERT): bidirectional; fill in masked tokens
- Seq2Seq (T5/BART): encode input; decode output autoregressively
- Temperature: scale logits before softmax — lower = more focused, higher = more random
- Top-k sampling: sample from k most probable tokens
- Top-p / Nucleus sampling: sample from tokens covering top p% probability mass
- Beam search: keep B best partial sequences at each step

19.2 Pretraining Objectives

Objective	Description
MLM (BERT)	Mask 15% tokens; predict from bidirectional context
CLM (GPT)	Predict next token; fully autoregressive
NSP (BERT)	Predict if sentence B follows sentence A
Span Masking (SpanBERT)	Mask contiguous spans; predict entire span
Replaced Token Detection (ELECTRA)	GAN-style; detect replaced tokens; efficient
Prefix LM (UniLM)	Mixed attention mask; both bidirectional + causal

19.3 RLHF & Instruction Tuning

- SFT (Supervised Fine-Tuning): finetune on (prompt, response) pairs
- Reward Model: train classifier to predict human preference between two responses
- PPO: optimize LLM to maximize reward model score with KL constraint
- DPO: Direct Preference Optimization — simpler than PPO, no reward model
- Constitutional AI (Claude): self-critique + revision to reduce harm

Chapter 20

MLOps, Deployment & Scaling

20.1 Training at Scale

Technique	Description
Data Parallelism	Replicate model; split batch; sync gradients (DDP)
Model Parallelism	Split model layers across GPUs (pipeline parallel)
Tensor Parallelism	Split weight matrices across GPUs (Megatron-LM)
ZeRO (DeepSpeed)	Shard optimizer state / gradients / params across GPUs
Gradient Checkpointing	Recompute activations in backward; saves memory
Mixed Precision (FP16/BF16)	Halve memory; faster; loss scaling for stability

20.2 Inference Optimization

- Quantization: INT8/INT4 weights — 2-8x speedup, ~1% accuracy drop
- Pruning: remove unimportant weights/heads; structured vs unstructured
- Knowledge Distillation: train small student to mimic large teacher
- ONNX: export PyTorch models for cross-platform deployment
- TensorRT: NVIDIA's inference optimizer; layer fusion, FP16/INT8
- vLLM: PagedAttention for LLM serving — continuous batching, high throughput
- KV Cache: cache key-value pairs during autoregressive decoding
- Speculative Decoding: draft small model; verify with large — 2-4x faster

20.3 ML Pipeline

- Data versioning: DVC, LakeFS — track dataset changes
- Experiment tracking: MLflow, Weights & Biases, TensorBoard
- Model registry: store, version, stage models (MLflow, HuggingFace Hub)
- Feature stores: Feast, Tecton — consistent features for training and serving
- CI/CD for ML: re-train trigger, automated eval, drift detection
- Monitoring: data drift (PSI, KL div), model drift (concept shift), outlier detection

20.4 Evaluation Metrics

Task	Key Metrics
Classification	Accuracy, Precision, Recall, F1, AUC-ROC, MCC
Detection	mAP@0.5, mAP@0.5:0.95, IoU
Segmentation	mIoU, Dice score, Pixel Accuracy

Generation (NLP)	BLEU, ROUGE, METEOR, BERTScore, Perplexity
Generation (Images)	FID, IS (Inception Score), LPIPS, SSIM
Regression	MSE, MAE, RMSE, R ² , MAPE

Interview Q&A;

Most Asked Deep Learning Interview Questions

Fundamentals

Q: What is the difference between deep learning and machine learning?

ML uses hand-crafted features fed into shallow models (SVM, linear regression, etc.). Deep learning automatically learns hierarchical feature representations from raw data via multi-layer neural networks. DL excels when data is high-dimensional (images, text, audio) and when enough labeled data exists.

Q: Explain the Universal Approximation Theorem.

A feedforward network with a single hidden layer containing sufficient neurons can approximate any continuous function to arbitrary precision on a compact domain, given a non-linear activation. This proves expressive power but says nothing about learnability or generalization. In practice, depth is more efficient than width.

Q: Why do we need non-linear activation functions?

Without non-linearity, stacking multiple linear layers is equivalent to a single linear transformation ($W_n \dots W_2 W_1 = W_{\text{combined}}$). Non-linear activations break this collapse, allowing networks to learn complex, non-linear decision boundaries and hierarchical representations.

Q: What is the vanishing gradient problem and how do you solve it?

During backprop in deep/recurrent networks, gradients are multiplied by many Jacobians. If activations saturate (sigmoid/tanh) or weights are small, these products shrink exponentially — gradients in early layers approach zero. Solutions: ReLU activations, batch normalization, residual connections (ResNets), LSTM gating, better initialization (He/Xavier), gradient clipping.

Q: What is overfitting? How do you detect and prevent it?

Overfitting = low training loss, high validation loss — model memorizes training data. Detect: monitor train vs val loss curves; val loss increases while train decreases. Prevent: dropout, L1/L2 regularization, data augmentation, early stopping, reduce model capacity, get more data.

Q: Explain bias-variance tradeoff.

Test error = Bias² + Variance + Noise. High bias = underfitting (model too simple). High variance = overfitting (model too complex). Regularization reduces variance at cost of bias. Ensembling reduces variance. More data reduces variance. Increasing model capacity reduces bias but increases variance.

Q: What is the difference between parameters and hyperparameters?

Parameters: learned from data (weights, biases). Hyperparameters: set before training (learning rate, batch size, number of layers, dropout rate, regularization strength). Parameters are optimized by backprop+optimizer. Hyperparameters are tuned by grid/random search, Bayesian optimization, or Hyperband.

Backpropagation & Optimization

Q: Explain backpropagation step by step.

1) Forward pass: compute predictions and loss. 2) Compute output layer gradient $dL/da[L]$. 3) Propagate backward using chain rule: $\delta[l] = (W[l+1]^T \delta[l+1]) * f'(z[l])$. 4) Compute parameter gradients: $dL/dW[l] = \delta[l] (a[l-1])^T$. 5) Update parameters: $W \leftarrow W - lr * dL/dW$. Automatic differentiation frameworks (PyTorch autograd) handle

this by recording operations in a computational graph.

Q: What is the difference between SGD, Adam, and AdamW?

SGD: simple gradient step; slow convergence; good generalization with LR scheduling. Adam: adaptive per-parameter learning rates using first+second moment estimates; fast convergence but can overfit. AdamW: fixes Adam's weight decay — decouples it from the adaptive learning rate update (wd applied to weights directly, not to the gradient update). AdamW is standard for transformers; SGD+momentum remains competitive for CNNs.

Q: Why does learning rate matter so much?

Too high: gradients explode, loss diverges or oscillates. Too low: very slow convergence, may get stuck in poor local minima. Optimal LR is task/architecture dependent. Techniques: LR range test, cosine annealing, warmup (transformers), cyclic LR. Rule of thumb: scale LR linearly with batch size (linear scaling rule).

Q: What is gradient clipping and when do you use it?

Clip gradient norm: if $\|g\| > \text{max_norm}$, scale $g = g * \text{max_norm} / \|g\|$. Used for RNNs (exploding gradient in BPTT), early training of large transformers, and any task with loss spikes. PyTorch: `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`. Standard `max_norm = 1.0` for LLMs.

Q: What is the difference between local minima and saddle points?

Local minima: all gradient directions point up — loss surrounded by higher values. Saddle points: gradient zero but some directions go up, some down — much more common in high-dimensional spaces (Dauphin et al.). Modern belief: local minima in deep nets are usually close to global minima in loss value; saddle points are more problematic. SGD noise helps escape both.

CNNs

Q: How does a convolutional layer differ from a fully connected layer?

Conv layer: local connectivity (each neuron sees only $K \times K$ region), weight sharing (same filter applied everywhere), translation equivariant. FC layer: all neurons connect to all inputs — no spatial structure, $O(n^2)$ parameters. Conv is much more parameter-efficient for spatially structured data like images.

Q: What is the receptive field of a neuron?

The region of the input that can influence a particular neuron's activation. Each conv layer adds $K-1$ to the effective receptive field. Stacking layers grows it: after L layers of kernel K , $RF = L*(K-1)+1$. Dilated convolutions expand RF without increasing params. Deeper networks have larger RF — can capture global structure.

Q: Explain how ResNet solves the degradation problem.

Without skip connections, adding more layers degrades training accuracy (not just overfitting — train error also increases). ResNet reformulates layers to learn residual $F(x) = H(x) - x$, so output = $F(x) + x$. If optimal is near identity, $F(x)=0$ is trivially learnable. Skip connections also provide gradient highways — gradient flows directly through the shortcut during backprop.

Q: What is depthwise separable convolution? Why is it efficient?

Factorizes standard convolution into: (1) Depthwise conv — $K \times K$ filter applied independently per channel: C_{in} filters. (2) Pointwise conv — 1×1 conv mixing channel info: $C_{in} * C_{out}$ filters. Parameter reduction: $DW_sep\ params = K^2 * C_{in} + C_{in} * C_{out}$ vs standard $K^2 * C_{in} * C_{out}$. For $K=3$: ~8-9x fewer params. Used in MobileNet, EfficientNet, Xception.

Q: What is Global Average Pooling (GAP) and why is it better than FC?

GAP takes the spatial average of each feature map — collapses $H \times W \times C$ to $1 \times 1 \times C$. Replaces large FC layers at end of CNN. Benefits: far fewer parameters, forces feature maps to be class-specific activation maps, more robust to spatial translations, enables GradCAM visualization, network accepts any input size.

RNNs & Sequence Models

Q: Why do vanilla RNNs struggle with long sequences?

BPTT multiplies Jacobians of hidden state transitions across T steps. If spectral radius of $W_{hh} < 1$, gradients vanish exponentially with distance. If > 1 , they explode. This makes learning long-range dependencies impractical. LSTMs solve this via gated cell state that can preserve information unchanged (gradient highway). Transformers solve it via direct attention to any position.

Q: Explain the LSTM gates and their roles.

Forget gate f_t : sigmoid output in $(0,1)$; controls what fraction of previous cell state C_{t-1} to keep. Input gate i_t : what new information to write to cell. Cell gate $C_{\tilde{t}}$: candidate values to write. Output gate o_t : what to expose from cell state as hidden state. The key insight: $C_t = f_t * C_{t-1} + i_t * C_{\tilde{t}}$ creates an additive (not multiplicative) update — gradient can flow unchanged through C_t .

Q: When would you use GRU vs LSTM?

GRU: fewer parameters, faster training, comparable performance on most tasks, better for smaller datasets or limited compute. LSTM: tends to outperform on tasks with very long sequences (>500 steps) where the separate cell state helps; better when you need explicit memory control. In practice, try both — the difference is often marginal. Both are largely replaced by transformers for NLP.

Attention & Transformers

Q: Explain scaled dot-product attention. Why the scaling?

$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) * V$. Q, K, V are linear projections of inputs. QK^T computes compatibility scores (dot products). Without scaling, for large d_k the dot products grow large, pushing softmax into saturation regions with tiny gradients. Dividing by $\sqrt{d_k}$ keeps variance ~ 1 regardless of dimension.

Q: What is multi-head attention and why use multiple heads?

Split Q, K, V into h heads; apply attention independently in each subspace; concatenate and project. Different heads learn to attend to different types of relationships simultaneously — some heads capture local syntactic dependencies, others long-range semantic, others coreference. Using $h=8$ heads with $d_k = d_{\text{model}}/h$ maintains the same total computation as single-head attention.

Q: What is the difference between encoder-only, decoder-only, and encoder-decoder transformers?

Encoder-only (BERT): bidirectional attention; see full context; best for understanding tasks (classification, NER, QA). Decoder-only (GPT): causal (left-to-right) masked attention; can only see past tokens; best for generation. Encoder-Decoder (T5, BART): encoder processes input with full attention; decoder generates output with causal attention + cross-attention to encoder — best for seq2seq tasks (translation, summarization).

Q: How does positional encoding work in transformers?

Attention is permutation-equivariant — it treats tokens as a set with no order. Positional encodings inject position information by adding sinusoidal or learned vectors to token embeddings. Sinusoidal PE: $PE(\text{pos}, 2i) = \sin(\text{pos}/10000^{(2i/d)})$. Modern alternatives: RoPE (rotary, in LLaMA), ALiBi (attention bias, for length extrapolation).

Q: What is FlashAttention and why does it matter?

FlashAttention (Dao et al., 2022) reformulates attention computation to be IO-aware: tiles Q,K,V to fit in SRAM (fast GPU cache), avoiding slow HBM reads/writes of the attention matrix. Result: 2-4x faster training, 5-20x less memory, mathematically identical output. Enables longer context windows and larger batch sizes. Standard in all modern LLM training.

Q: Explain LoRA fine-tuning.

LoRA (Hu et al.) keeps pretrained weights W frozen and learns a low-rank decomposition of the weight update: $W' = W + AB$, where $A \in \mathbb{R}^{(d \times r)}$, $B \in \mathbb{R}^{(r \times k)}$, $r \ll \min(d,k)$. Only A and B are trained — typically 0.1% of original

parameters. At inference, merge: $W' = W + AB$ — zero latency cost. QLoRA extends this by quantizing W to 4-bit NF4, enabling fine-tuning large LLMs on consumer GPUs.

Regularization & Normalization

Q: How does dropout work during training vs inference?

Training: each neuron independently set to 0 with probability p ; remaining neurons scaled by $1/(1-p)$ [inverted dropout]. Each forward pass = different thinned network — ensemble effect. Inference: dropout disabled; all neurons active. The $1/(1-p)$ scaling ensures expected activation values are the same at train and test time. Typical $p=0.5$ for FC, 0.1-0.2 for conv.

Q: Why does batch normalization help?

1) Reduces internal covariate shift: normalizes layer inputs, making each layer less dependent on previous layer's distribution — faster, more stable training. 2) Allows higher learning rates. 3) Slight regularization effect from mini-batch statistics noise. 4) Reduces sensitivity to initialization. Trainable gamma/beta allow network to recover any desired distribution.

Q: When should you use Layer Norm vs Batch Norm?

Batch Norm: CNNs with large batches (≥ 32); spatially structured data. Doesn't work for small batches, online learning, or variable-length sequences. Layer Norm: Transformers and RNNs; independent of batch size; normalizes per sample across feature dimension. RMSNorm (Layer Norm without mean centering) used in LLaMA — simpler, similar performance.

Generative Models

Q: Explain the KL divergence term in VAE.

$ELBO = E[\log p(x|z)] - KL(q(z|x)||p(z))$. The KL term forces the approximate posterior $q(z|x)$ toward the standard normal prior $N(0, I)$. Without it: encoder collapses to delta functions (perfect encoding but no generalization, no sampling). KL term acts as a regularizer on the latent space — makes it smooth and continuous, enabling interpolation and sampling of new data.

Q: What is mode collapse in GANs and how do you fix it?

Mode collapse: generator learns to produce a small subset of realistic samples (ignoring the full data distribution) that reliably fool the discriminator. Fixes: (1) Wasserstein GAN — use Wasserstein distance instead of JS divergence, more informative gradient. (2) Minibatch discrimination — discriminator sees batches, penalizes lack of diversity. (3) Spectral normalization. (4) Progressive growing (StyleGAN). (5) Training tricks: different LR for G/D, gradient penalties.

Q: What is the difference between VAE and GAN?

VAE: encoder-decoder; explicit latent space; optimizes ELBO; smooth interpolation; blurry outputs; fast stable training. GAN: implicit density; adversarial training; no encoder (standard); sharper samples; unstable training; mode collapse risk. Diffusion models have surpassed both for image quality.

Practical / System Design

Q: How do you handle class imbalance in a deep learning classifier?

1) Resampling: oversample minority (SMOTE) or undersample majority. 2) Class weights in loss: $weight = N_{total} / (N_{classes} * N_{class_i})$. 3) Focal loss: down-weights easy examples (well-classified majority), focuses on hard minority examples. 4) Threshold tuning post-training: choose threshold maximizing F1 or business metric. 5) Ensemble methods. 6) Data augmentation specifically for minority class.

Q: You have 100K images and no labels. How do you build a classifier?

Self-supervised pretraining approach: 1) Pretrain with contrastive SSL (SimCLR, DINO, MoCo) on all 100K images. 2) Label a small subset (1-5K) manually. 3) Fine-tune the SSL backbone with a linear probe or full fine-tuning on labeled subset. Alternatively, use a large pretrained model (CLIP, DINO-ViT) directly — zero-shot or few-shot classification without any labels.

Q: Explain knowledge distillation.

Train a small student model to mimic a large teacher model. Student learns from: (1) soft labels = teacher's probability outputs (temperature $T > 1$ to soften distribution — dark knowledge). (2) Intermediate feature matching — student's hidden activations match teacher's. Loss: $\alpha * \text{CE}(\text{student}, \text{hard labels}) + (1-\alpha) * \text{KL}(\text{student_soft}, \text{teacher_soft})$. DistilBERT: 6-layer student from 12-layer BERT, 40% smaller, 60% faster, 97% performance.

Q: How do you choose batch size? What are the effects?

Larger batch: faster GPU utilization, more stable gradient estimates, but may generalize worse (sharp minima). Smaller batch: noisier gradients, regularization effect, often finds flatter minima (better generalization). Linear scaling rule: if batch 2x, LR 2x. Practical range: 32-256 for CNNs; 2048-4096 for transformer pretraining; 1-16 for fine-tuning LLMs. Critical batch size: below this, increasing batch gives no speedup.

Q: What is gradient accumulation and when do you use it?

Simulate large batch on limited GPU memory: compute gradients for N small batches, accumulate (sum) them, then do one optimizer step. Effective batch = $\text{small_batch} * N_{\text{accumulation_steps}}$. Essential for transformer fine-tuning with limited VRAM. PyTorch: only call `optimizer.step()` and `zero_grad()` every N steps. Note: BN statistics still computed per actual (small) batch — inconsistency vs true large batch.

Q: How do you debug a neural network that isn't learning?

Systematic debugging: 1) Overfit a single batch first — model should reach near-zero loss. 2) Check learning rate — try $1e-3$ (Adam) or $1e-2$ (SGD). 3) Verify loss function and labels are correct. 4) Check for NaN/inf in activations and gradients. 5) Plot loss curve — is it decreasing at all? 6) Gradient norm — too small (vanishing) or too large (exploding)? 7) Visualize predictions on training data. 8) Check data pipeline — ensure augmentation isn't destroying labels.

Q: Explain transfer learning from ImageNet to a medical imaging task.

Two-phase training: Phase 1 — freeze backbone (ResNet/EfficientNet pretrained on ImageNet), train only new classifier head for 5-10 epochs with high LR ($1e-3$). Phase 2 — unfreeze all layers, train end-to-end with low LR ($1e-5$ to $1e-4$) and LR decay. Key considerations: medical images differ from natural images (grayscale, different textures) — may need to fine-tune from early layers. Use aggressive augmentation (flips, rotations, elastic deformation). Class imbalance common — use weighted loss or focal loss.

Advanced Topics

Q: What is the difference between `model.eval()` and `torch.no_grad()`?

`model.eval()`: changes model behavior — disables dropout (all neurons active), switches BN to use running stats instead of batch stats. Does NOT disable gradient computation. `torch.no_grad()`: disables gradient tape — operations don't create computation graph nodes, saving memory and compute. For inference: use BOTH together. `model.eval()` for correct predictions; `torch.no_grad()` for efficiency.

Q: Explain the attention mechanism in transformers from scratch.

Think of attention as a soft database lookup. Query Q asks a question. Keys K are what each position knows about itself. Dot product QK^T measures query-key compatibility. Softmax converts to attention weights (sum to 1). These weights combine Values V — positions with high compatibility contribute more to output. This is fully differentiable and parallelizable. Self-attention = Q, K, V all from same sequence. Cross-attention = Q from one sequence, K, V from another.

Q: What are the limitations of transformers?

1) Quadratic complexity $O(n^2)$ in sequence length — limits context window. 2) Position encoding limits length generalization. 3) No built-in recurrence — need techniques like RoPE, ALiBi for length extrapolation. 4) Data hungry — needs massive pretraining. 5) Memory intensive — KV cache grows with context. 6) Computationally expensive to train. Mitigations: efficient attention (FlashAttention), sparse attention, state space models (Mamba as alternative), context compression.

Q: What is the difference between precision and recall? When do you optimize which?

Precision = $TP/(TP+FP)$: of all predicted positives, how many are truly positive — measures false alarm rate. Recall = $TP/(TP+FN)$: of all actual positives, how many did we catch — measures miss rate. Optimize precision when: false positives are costly (spam filter, fraud detection — don't want to bother real emails/transactions). Optimize recall when: false negatives are costly (cancer screening, safety systems — can't miss a real case). F1 = harmonic mean, balanced tradeoff.

Q: What is gradient tape / automatic differentiation?

Modern DL frameworks use reverse-mode autodiff. Forward pass: record all operations in a computational graph (tape). Backward pass: traverse graph in reverse, applying chain rule to accumulate gradients. PyTorch uses dynamic graph (tape built each forward call). TF1 used static graph (build once, run many). Benefits vs symbolic differentiation: works on arbitrary Python control flow; vs numerical differentiation: exact and $O(1)$ cost in #outputs regardless of #inputs.

Q: Explain RLHF (Reinforcement Learning from Human Feedback).

Used to align LLMs with human preferences: Step 1 SFT — fine-tune LLM on high-quality (prompt, response) demonstrations. Step 2 Reward Model — collect human preference data (A vs B comparisons); train classifier to predict which response is better. Step 3 PPO — use RL (PPO algorithm) to fine-tune LLM to maximize reward model score, with KL divergence penalty to prevent drifting too far from SFT model. Result: models like ChatGPT, Claude that follow instructions and are helpful/harmless. DPO is a simpler alternative that avoids explicit RL.

Quick Reference

Formulas, Architectures & Rules of Thumb

Must-Know Formulas

Formula / Concept	Expression
Attention	$\text{softmax}(QK^T/\sqrt{d_k}) V$
Adam update	$w \leftarrow lr * m_hat / (\sqrt{v_hat} + \epsilon)$
Backprop delta	$\text{delta}[l] = (W[l+1]^T \text{delta}[l+1]) * f'(z[l])$
Cross-entropy	$L = -\sum y_c \log(p_c)$
ELBO (VAE)	$E[\log p(x z)] - KL(q(z x) p(z))$
BN	$(x - \mu) / \sqrt{\sigma^2 + \epsilon} * \gamma + \beta$
IoU	$\text{Area}(\text{Pred} \ \& \ \text{GT}) / \text{Area}(\text{Pred} \ \ \text{GT})$
Perplexity	$\exp(-\frac{1}{N} \sum \log P(w_i \text{context}))$
InfoNCE	$-\log[\exp(\text{sim}(z_i, z_j) / \tau) / \sum \exp(\text{sim}(z_i, z_k) / \tau)]$
LoRA	$W' = W + AB, r \ll \min(d, k)$
Output size	$\text{floor}((W - K + 2P) / S) + 1$
He init var	$2 / \text{fan_in}$

Architecture Decision Guide

Task	Recommended Architecture
Image classification	EfficientNetB0-B4 or ResNet50 pretrained
Object detection	YOLOv8 (real-time) or DETR/Co-DETR (SOTA)
Semantic segmentation	SegFormer or DeepLabV3+
Text classification	DistilBERT / RoBERTa fine-tuned
Text generation	GPT-2 small / LLaMA 7B fine-tuned
Seq2seq (NLP)	T5-small or BART
Time series	LSTM + attention or PatchTST
Tabular data	XGBoost/LightGBM first; TabNet for DL
Image generation	Stable Diffusion (latent diffusion)
Representation learning	DINO ViT-B/16 or CLIP

Hyperparameter Rules of Thumb

- Learning rate: $1e-4$ to $1e-3$ for Adam; $1e-2$ to 0.1 for SGD+momentum
- Batch size: 32-256 for CNN; 16-64 for fine-tuning transformers
- Dropout: 0.5 for FC layers; 0.1-0.2 for conv; 0.0-0.1 for transformers
- Weight decay: $1e-4$ for CNN; 0.01-0.1 for transformers (AdamW)
- Warmup steps: 1-10% of total training steps for transformers
- LoRA rank r : 4-16 for fine-tuning; start with $r=8$, $\alpha=16$
- Number of epochs: 10-50 for fine-tuning; use early stopping
- Gradient clip max norm: 1.0 for LLMs/RNNs; often not needed for CNNs